[승코의 차근차근 Flutter] 08, Database(Sqflite)(1)

✅ `sqflite` 문법 완전 정복 가이드

✅ 1. 설치 및 import

```
dependencies:
  sqflite: ^2.3.2
  path: ^1.9.0
```

```
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart'; // 경로 조작에 사용
```

> path는 join()으로 경로 연결을 도와줌. OS에 따라 /, \ 구분해 줌.

✅ 2. 데이터베이스 열기 & 생성

```
Future<Database> _initDatabase() async {
  String dbPath = await getDatabasesPath(); // 내부 저장소 경로
  String path = join(dbPath, 'todo.db');

  return await openDatabase(
    path,
    version: 1,
    onCreate: _onCreate, // DB 처음 생성 시 호출됨
  );
}
```

🔷 `openDatabase()` 주요 인자

인자명	설명
`path`	데이터베이스 경로 (`join()`으로 OS 호환)
`version`	데이터베이스 버전 (업데이트용)
`onCreate`	테이블 최초 생성 시 호출
`onUpgrade`	버전 변경 시 호출
`onOpen`	열릴 때마다 호출됨

✅ 3. 테이블 생성 (`onCreate`)

```
Future<void> _onCreate(Database db, int version) async {
  await db.execute('''
    CREATE TABLE todos (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      title TEXT NOT NULL,
      isDone INTEGER NOT NULL
    )
  ''');
}
```

🔷 SQL 문법 요약

키워드	설명
`CREATE TABLE`	테이블 생성
`INTEGER PRIMARY KEY`	정수형 기본키 (자동 증가)
`AUTOINCREMENT`	자동 증가 옵션
`TEXT`	문자열 타입
`NOT NULL`	null 금지

> isDone은 SQLite에 boolean이 없기 때문에 INTEGER (0 또는 1)로 저장

✅ 4. 데이터 삽입 (Insert)

```
Future<int> insertTodo(Todo todo) async {
  final db = await database;

  return await db.insert(
    'todos',            // 테이블명
    todo.toMap(),       // 저장할 값 Map 형태
    conflictAlgorithm: ConflictAlgorithm.replace, // id 중복 시 덮어쓰기
  );
}
```

🔷 `insert()` 문법

```
Future<int> insert(
  String table,
  Map<String, Object?> values, {
  String? nullColumnHack,
  ConflictAlgorithm? conflictAlgorithm,
})
```

- **반환값:** `id` (자동 생성된 기본키)

✅ 5. 데이터 조회 (Select)

```
Future<List<Todo>> getTodos() async {
  final db = await database;

  final List<Map<String, dynamic>> maps = await db.query('todos');

  return maps.map((map) => Todo.fromMap(map)).toList();
}
```

🔷 `query()` 문법

```
Future<List<Map<String, dynamic>>> query(
  String table, {
  bool? distinct,
  List<String>? columns,
  String? where,
  List<Object?>? whereArgs,
  String? orderBy,
  int? limit,
  int? offset,
})
```

🔷 예시: 조건 조회

```
final result = await db.query(
  'todos',
  where: 'isDone = ?',
  whereArgs: [0], // 미완료 항목만 조회
);
```

> whereArgs는 SQL 인젝션을 방지하므로 직접 값을 문자열에 넣지 말고 ? 사용!

✅ 6. 데이터 수정 (Update)

```
Future<int> updateTodo(Todo todo) async {
  final db = await database;

  return await db.update(
    'todos',
    todo.toMap(),
    where: 'id = ?',
    whereArgs: [todo.id],
  );
}
```

🔷 `update()` 문법

```
Future<int> update(
  String table,
  Map<String, Object?> values, {
  String? where,
  List<Object?>? whereArgs,
  ConflictAlgorithm? conflictAlgorithm,
})
```

- 반환값: 영향을 받은 행의 수 (`int`)

✅ 7. 데이터 삭제 (Delete)

```
Future<int> deleteTodo(int id) async {
  final db = await database;
```

```
  return await db.delete(
    'todos',
    where: 'id = ?',
    whereArgs: [id],
  );
}
```

🔷 `delete()` 문법

```
Future<int> delete(
  String table, {
  String? where,
  List<Object?>? whereArgs,
})
```

✅ 8. raw 쿼리 사용 (고급)

- 직접 SQL문을 쓰고 싶을 때 사용

```
await db.rawInsert('INSERT INTO todos (title, isDone) VALUES (?, ?)', ['책 읽기', 0]);

List<Map> result = await db.rawQuery('SELECT * FROM todos WHERE isDone = ?', [1]);

await db.rawUpdate('UPDATE todos SET isDone = ? WHERE id = ?', [1, 3]);

await db.rawDelete('DELETE FROM todos WHERE isDone = ?', [1]);
```

> rawX 계열은 SQL을 정확히 알고 쓸 수 있을 때만 사용 권장.

✅ 9. 데이터 모델 (Map 변환)

클래스 → Map (저장 시)

```
Map<String, dynamic> toMap() => {
  'id': id,
  'title': title,
  'isDone': isDone ? 1 : 0, // bool → int
};
```

Map → 클래스 (조회 시)

```
factory Todo.fromMap(Map<String, dynamic> json) => Todo(
  id: json['id'],
  title: json['title'],
  isDone: json['isDone'] == 1,
);
```

✅ 10. 데이터베이스 경로 확인 & 삭제

```
String dbPath = await getDatabasesPath();
print("DB 경로: $dbPath");

String path = join(dbPath, 'todo.db');
await deleteDatabase(path); // 초기화 용
```

✅ 11. 실전에서의 모범 구조

📁 구조 예시

```
lib/
 ├ models/
 │ └ todo.dart
 ├ database/
 │ └ database_helper.dart
```

```
├ providers/
│ └ todo_provider.dart
├ screens/
│ └ todo_list_screen.dart
```

> 기능별로 역할 분리: DB 관련은 database_helper.dart에서만 처리.

✅ 12. 자주 하는 실수 & 팁

실수/문제	원인/해결
`null` 오류	async 처리 안했거나 `await` 누락
DB 중복 생성	`singleton 패턴` 사용 필수
UI 반영 안 됨	`Provider` 또는 `setState()`로 상태 반영 필요
테이블 구조 바꾸면 반영 안됨	`version` 증가 + `onUpgrade()` 필요
rawQuery 인젝션 위험	`?` + `whereArgs` 필수 사용

✅ 요약: `sqflite` 핵심 문법 정리

작업	메서드	예시
생성	`db.execute('CREATE TABLE ...')`	테이블 생성
삽입	`db.insert()`	`db.insert('todos', todo.toMap())`
조회	`db.query()`	`db.query('todos')`
수정	`db.update()`	`db.update('todos', todo.toMap(), where: ...)`
삭제	`db.delete()`	`db.delete('todos', where: ...)`
고급 쿼리	`db.rawQuery()` 등	SQL 직접 작성 가능

📌 참고용: 전체 흐름 요약

```
// 1. db 객체 초기화 (singleton)
final db = await DatabaseHelper.instance.database;
```

```
// 2. todo 객체 생성
Todo todo = Todo(title: "공부하기");

// 3. insert
await db.insert('todos', todo.toMap());

// 4. select
List<Map<String, dynamic>> maps = await db.query('todos');

// 5. update
todo.isDone = true;
await db.update('todos', todo.toMap(), where: 'id = ?', whereArgs: [todo.id]);

// 6. delete
await db.delete('todos', where: 'id = ?', whereArgs: [todo.id]);
```